

LLM Benchmark Report

Open-Source Model Evaluation for Developer Tooling

Author	Madeleina Zegeye
Date	June 26, 2025
Environment	Azure VM: 3 runs per model per prompt
Models Evaluated	Mistral 7B, DeepSeek, Qwen, Code Llama
Purpose	Select best open-source LLM for Azure-hosted developer tooling, Cursor IDE integration, and internal chatbot support

Section 1: Test Prompts and What They Measure

Each prompt was selected to test a distinct capability relevant to developer tooling: algorithmic coding, conceptual explanation, mathematical reasoning, real-world software engineering, and general conversational ability.

#	Prompt	What It Tests
1	Write a Python function to reverse a linked list.	Algorithmic coding: data structures, pointer logic, clean implementation
2	Explain what a closure is in JavaScript.	Conceptual explanation: accuracy, clarity, use of examples
3	If a train travels 60 miles in 1.5 hours, what is its average speed?	Basic reasoning and math: correct answer, no hallucination, directness
4	Write a function that connects to a database, queries all users, and returns the result as a JSON object.	Real-world software engineering: practical code, error handling, usable output
5	Hi there! What can you do?	Conversational ability: self-awareness, coherence, appropriate casual response

Average Response Times

Model	Typical Response Time
Mistral	10.7s (math prompt), 84–179s (other prompts)
DeepSeek	164–170s consistently across all prompts
Code Llama	170–183s consistently across all prompts
Qwen	84–184s depending on prompt complexity

Section 2: Model Responses by Prompt

Each model was run 3 times per prompt on the Azure VM. Where responses were consistent across runs, one representative output is shown.

Prompt 1: Write a Python function to reverse a linked list.

DeepSeek

```
class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def reverse(self):
        prev = None
        cur = self.head
        while cur:
            ... [response truncated at 732 chars]
```

Avg response time: ~169s avg | Output length: 732 chars

PASS: Correct implementation

PASS: Functional iterative logic

PASS: Full LinkedList class included

Qwen

```
def reverse_linked_list(head):
    prev_node = None
    current_node = head
    while current_node is not None:
        next_temp = current_node.next # Store next node
        current_node.next = prev_node # Reverse the pointer
        prev_node = current_node
        current_node = next_temp
    return prev_node
```

[Explanation of linked list structure included before code. Consistent across runs.]

Avg response time: ~183s avg | Output length: 838–942 chars

PASS: Correct implementation

PASS: More explanatory: explained structure before coding

Mistral

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_linked_list(head: ListNode) -> ListNode:
    prev_node = None
    current_node = head
```

```
while current_node is not None:
    next_node = current_node.next # Save the next node
    current_node.next = prev_node # Reverse the link
    prev_node = current_node      # Move to next pair
    current_node = next_node
return prev_node
```

Avg response time: ~135s avg | Output length: 545 chars

PASS: Correct

PASS: Concise with type hints

Code Llama

Example:

Input: 1->2->3->4->5->NULL

Output: 5->4->3->2->1->NULL

Follow up:

A linked list can be reversed either iteratively or recursively.

Could you implement both?

[Note: Returned problem description and examples only: no implementation provided]

Avg response time: ~173s avg | Output length: 357 chars

NOTE: Did not implement a solution

NOTE: Returned problem description only

Prompt 2: Explain what a closure is in JavaScript.**DeepSeek**

A closure is a function that has access to its own scope, the outer function's scope, and the global scope. Closures are created every time a function is created.

A closure has three lexical scopes:

1. Own scope: where the closure was created.
2. Outer function scope: the enclosing function's scope.
3. Global scope: the global script environment.

```
function outerFunction() {
  let outerVariable = 'I am outside!';
  function innerFunction() { ... } [truncated at 866 chars]
```

Avg response time: ~167s avg | Output length: 866 chars

PASS: Correct

PASS: Three-scope breakdown included

PASS: Working code example provided

Qwen

A closure is a feature in JavaScript that allows a function to access and manipulate variables from its parent scope, even after the parent function has finished executing.

```
function outerFunction() {
  var outerVariable = 'I am from outer function';
  function innerFunction() { console.log(outerVariable); }
  return innerFunction;
}
var innerFunc = outerFunction();
innerFunc(); // Outputs 'I am from outer function'
```

Avg response time: ~181s avg | Output length: 963–1005 chars

PASS: Strongest explanation overall

PASS: Clear definition with retained scope discussion

PASS: Clean working example

Mistral

A closure is a function in JavaScript that has access to the outer function's variables, even after the outer function has returned. This is possible because a closure is defined within another function, retaining access to its variables. Closures are often used to create private variables and implement higher-order functions.

Avg response time: ~84s avg | Output length: 496 chars

PASS: Correct

NOTE: Less detailed than Qwen or DeepSeek

Code Llama

```
function makeAdder(num) {
  return function(otherNum) { return num + otherNum; };
}
const add5 = makeAdder(5);
console.log(add5(2)); // 7
```

```
### Explain how function scope works in JavaScript.  
### Explain how the `this` keyword works in JavaScript.  
[Continued adding unrelated JS topics across all 3 runs]
```

Avg response time: ~173s avg | Output length: 527 chars

NOTE: Provided code example but no verbal explanation

NOTE: Prompt drift into unrelated JavaScript topics

Prompt 3: If a train travels 60 miles in 1.5 hours, what is its average speed?

Correct answer: 40 mph

Model	Correct?	Response Time	Notes
DeepSeek	Yes	~166–169s	Provided formula + full calculation
Qwen	Yes	~84–120s	Formatted with mathematical formula notation
Mistral	Yes	~10.7s	Direct answer: fastest by a significant margin
Code Llama	No	~172s	Looped the prompt ~9 times with no answer given

Code Llama output (consistent across all 3 runs): "A train travels 60 miles in 1.5 hours. What is its average speed?": repeated approximately 9 times with no answer provided.

Prompt 4: Write a function that connects to a database, queries all users, and returns the result as a JSON object.

DeepSeek

```
import psycopg2, json

def get_users():
    try:
        connection = psycopg2.connect(
            user='username', password='password',
            host='localhost', port='5432', database='database_name'
        )
        cursor = connection.cursor()
        cursor.execute('SELECT * FROM users')
        rows = cursor.fetchall()
        users = [{'id': row[0], 'name': row[1], 'email': row[2]} for row in rows]
        return json.dumps(users)
    except (Exception, psycopg2.Error) as error: ... [truncated at 797 chars]
```

Avg response time: ~168s avg | Output length: 797 chars

PASS: Practical solution

PASS: Python/psycopg2: real-world choice

PASS: JSON serialization included

Qwen

```
def get_users_with_pagination(page_size: int, page_number: int):
    conn = sqlite3.connect('your_database.db')
    cursor = conn.cursor()
    query = 'SELECT * FROM users LIMIT ? OFFSET ?'
    offset = (page_number - 1) * page_size if page_number > 0 else 0
    cursor.execute(query, (page_size, offset))
    ...

[Also added: pagination, auth, rate limiting, and credential validation across all 3 runs]
```

Avg response time: ~182s avg | Output length: 1026–1240 chars

PASS: Most advanced response

PASS: Added pagination, auth, and rate limiting beyond the prompt requirements

Mistral

```
const mysql = require('mysql');

const connection = mysql.createConnection({
  host: 'localhost', user: 'your_database_user',
  password: 'your_database_password', database: 'your_database_name'
});

connection.connect((err) => {
  if (err) throw err;
  connection.query('SELECT * FROM users', (error, results) => {
    const users = results.map((user) => ({ id: user.id, name: user.name, email:
user.email }));
    // Return the JSON object... [truncated at 778 chars]
  });
});
```

Avg response time: ~179s avg | Output length: 778 chars

PASS: Correct working solution

PASS: Used JavaScript/MySQL: valid, assumes language preference

Code Llama

```
function getUsers(db) { // Your code here }

### 2. Write a function that takes a JSON object and returns a string of HTML.
function renderUsers(users) { // Your code here }
### 3. [above repeated 3 more times]
```

[Note: Returned empty stubs only. No working code produced across all 3 runs.]

Avg response time: ~172s avg | Output length: 736 chars

FAIL: Failed task: no working implementation

FAIL: Prompt drift into unrelated functions

Prompt 5: Hi there! What can you do?**DeepSeek**

```
1. /start - Start the bot
2. /help - Get help
3. /add - Add a new task
4. /list - List all tasks
5. /delete - Delete a task
...
Made by @mohamed_abdul_hakim
Source code: https://github [truncated]
```

[Note: Appears to be a Telegram bot persona, not a general assistant response]

Avg response time: ~168s avg | Output length: 658 chars

NOTE: Relevant format but hallucinated a specific bot persona

NOTE: Not a general-purpose assistant response

Qwen

Hi there! What can you do?

[Response was a 26-character echo of the input only: no content generated across all 3 runs]

Avg response time: ~2.8s avg | Output length: 26 chars

FAIL: Complete failure: no response content generated

Mistral

I'm an AI language model that can help you with a variety of tasks:

1. Answer questions on a wide range of topics.
2. Provide information on news, weather, sports, and more.
3. Help with tasks, from finding information to completing forms.
4. Provide recommendations on products, services, and more.

Avg response time: ~179s avg | Output length: 881 chars

PASS: Best conversational response

PASS: Coherent, appropriate, and helpful

Code Llama

```
Comment: @JonathanLeffler: I'm not sure what you mean by 'there'.
I'm not sure what you mean by 'there'. I'm not sure what you mean by 'there'.
... [phrase repeated approximately 15 times across all 3 runs]
```

Avg response time: ~172s avg | Output length: 666 chars

FAIL: Complete failure

FAIL: Incoherent repetitive output

Section 3: Analysis and Final Ranking

Mistral 7B

Strengths

- Fastest response times: answered the math prompt in 10.7s vs 165–172s for all other models
- Clean, well-commented code on Prompt 1 with type hints included
- Accurate closure explanation on Prompt 2 without unnecessary padding
- Best conversational response on Prompt 5: coherent and appropriate
- Most consistent output across all 3 runs: near-identical responses each time

Weaknesses

- Database function (Prompt 4) defaulted to JavaScript/MySQL rather than Python: valid but assumes a language preference
- Closure explanation was less detailed than Qwen or DeepSeek

Summary: Mistral delivered the most reliable, production-ready output across the benchmark. Speed, consistency, and appropriate conversational behavior were its strongest traits. Best fit for developer tooling.

DeepSeek

Strengths

- Solid linked list implementation: full LinkedList class with helper methods
- Accurate closure explanation with three-scope lexical breakdown
- Correct math answer with clear formula and explanation
- Practical database function using Python/psycopg2

Weaknesses

- Response times uniformly high (~165–172s) regardless of task complexity: even simple math took 166s
- Prompt 5 produced a Telegram bot persona rather than a general assistant response
- Responses echoed the input prompt before answering, adding noise
- Prompt 1 and 4 outputs were truncated: full implementations not confirmed

Summary: DeepSeek showed strong coding and reasoning ability but struggled with conversational prompts and had uniformly slow response times. Strong second-tier candidate for code-focused workloads.

Qwen

Strengths

- Strongest explanation quality on Prompt 2: clear definition with a proper JavaScript example
- Correct and well-formatted answer on Prompt 3 with formula notation
- Prompt 4 was the most advanced response: added pagination, auth, and rate limiting
- Correct linked list implementation on runs 2 and 3

Weaknesses

- Prompt 5 failed completely: output was a 26-character echo of the input across all 3 runs
- Prompt 1 run 1 provided a scaffold with placeholder code rather than a working implementation

- Slowest model overall for most prompts (~180–185s avg)

Summary: Qwen showed strong technical explanation and reasoning ability but the complete failure on conversational input is a significant limitation for any tooling involving user interaction.

Code Llama

Strengths

- Consistent timing across prompts (~170–177s)
- Prompt 2 provided a valid closure code example, showing some conceptual understanding

Weaknesses

- Prompt 1: Returned problem description only. No implementation produced across all 3 runs
- Prompt 2: Drifted into unrelated JavaScript topics (function scope, 'this' keyword)
- Prompt 3: Looped the prompt ~9 times with no answer. Critical failure
- Prompt 4: Returned empty function stubs with placeholder comments. No working code
- Prompt 5: Repeated a single incoherent phrase ~15 times. Complete failure

Summary: Code Llama was the weakest performer across all categories. Critical failures on 3 of 5 prompts and incomplete output on the remaining two make it unsuitable for unsupervised developer tooling.

Quantitative Ranking

Scores reflect benchmark results only (scale: 1–10 per category).

Rank	Model	Coding	Reasoning	Chat	Consistency	Total
1	Mistral	9/10	9/10	10/10	10/10	38/40
2	DeepSeek	9/10	9/10	6/10	10/10	34/40
3	Qwen	8/10	9/10	2/10	8/10	27/40
4	Code Llama	4/10	2/10	1/10	3/10	10/40

Final Recommendation

This benchmark evaluated models against the following project requirements:

- Local deployment on Azure VM
- Cursor IDE integration
- Agentic developer workflows
- Coding assistance
- Internal chatbot support

Model	Recommendation	Rationale
Mistral 7B	Recommended	Only model to handle coding, reasoning, and conversational tasks without major failures. Fastest response times across the board.
DeepSeek	NOTE: Secondary option	Strong coder and reasoner, but slow response times and chatbot persona failure limit suitability for general use.
Qwen	NOTE: Limited use cases	Strong at explanation and code tasks, but complete conversational failure disqualifies it for chatbot or interactive use.
Code Llama	Not recommended	Failed or produced incomplete output on all 5 prompts. Not suitable for unsupervised use in current configuration.

Mistral 7B Instruct was the strongest overall performer because it was the only model that consistently handled coding tasks, reasoning tasks, and conversational tasks without major hallucinations or prompt drift. DeepSeek finished a close second and was arguably the strongest pure coding model. Qwen showed strong reasoning and coding but weak chatbot behavior. Code Llama significantly underperformed the other three models in this benchmark.